



[데이터 수집용](https://docs.google.com/spreadsheets/d/1YIJ18eApWzd3qbpds8q4-yrFn5i6GgGfiDymwUR8734/edit?gid=0#gid=0) [분석 결과 공유](https://docs.google.com/presentation/d/1WZ-l75mrvAapqYKdPfsYUPnnDzMqVdsJxnmaGf4fLPY/edit?usp=sharing)

#1 데이터 전처리 및 탐색

```
In [ ]: import matplotlib.pyplot as plt
import matplotlib.patches as patches
import numpy as np
import io
import pandas as pd
```

데이터 수집

```
In [ ]: df = pd.read_csv(io.StringIO('''
ID,키,발 사이즈,"성별(남,여)"
차은우,167,250,여
홍지후,162,245,여
"—"—,164,255,여
  ll,168,240,여
장윤아,162,240,여
수혜니,165,230,여
미미민지,158,230,여
김노겸,159,240,여
애기공주,198,320,여
집가고싶다고,167,230,여
ㅇ,186,280,남
ㅇ[]ㅇ,161,250,여
송재민,184,290,남
정세현,174,270,남
권하울,170,270,남
박지원,181,300,남
류민석,151,220,여
악,156,245,여
이세한,176,265,남
ㅇㅅㅇ,162,245,남
이연지,165,245,여
어니ㅏㅇ머,162,250,여
윤하준,175,265,상남자
이름,176,270,남
'''))
df
```

	ID	키	발 사이즈	성별(남,여)
0	차은우	167	250	여
1	홍지후	162	245	여
2	—"—	164	255	여
3	ll	168	240	여
4	장윤아	162	240	여
5	수혜니	165	230	여
6	미미민지	158	230	여
7	김노겸	159	240	여
8	애기공주	198	320	여
9	집가고싶다고	167	230	여
10	ㅇ	186	280	남

	ID	키	발 사이즈	성별(남,여)
11	ㅇ[]ㅇ	161	250	여
12	송재민	184	290	남
13	정세헌	174	270	남
14	권하울	170	270	남
15	박지원	181	300	남
16	류민석	151	220	여
17	악	156	245	여
18	이세한	176	265	남
19	ㅇㅅㅇ	162	245	남
20	이연지	165	245	여
21	어니ㅏㅇ머	162	250	여
22	윤하준	175	265	상남자
23	이름	176	270	남

데이터 전처리

한국어 -> 영어

```
In [ ]: df.columns = ['ID','height','shoe size','sex']
```

In []:

df

	ID	height	shoe size	sex
0	차은우	167	250	여
1	홍지후	162	245	여
2	—"—	164	255	여
3	ll	168	240	여
4	장윤아	162	240	여
5	수혜니	165	230	여
6	미미민지	158	230	여
7	김노겸	159	240	여
8	애기공주	198	320	여
9	집가고싶다고	167	230	여
10	ㅇ	186	280	남
11	ㅇ[]ㅇ	161	250	여
12	송재민	184	290	남
13	정세헌	174	270	남
14	권하울	170	270	남
15	박지원	181	300	남
16	류민석	151	220	여
17	악	156	245	여
18	이세한	176	265	남
19	ㅇㅅㅇ	162	245	남
20	이연지	165	245	여
21	어니ㅏㅇ머	162	250	여
22	윤하준	175	265	상남자
23	이름	176	270	남

결측치 처리

```
In [ ]: gender_map = {
        '남': 'male',
        '상남자': 'male', # '상남자'도 'male'로 매핑
        '여': 'Female'
    }
    df['sex'] = df['sex'].map(gender_map)
    df
```

	ID	height	shoe size	sex
0	차은우	167	250	Female
1	홍지후	162	245	Female
2	—"—	164	255	Female
3	ll	168	240	Female
4	장윤아	162	240	Female
5	수혜니	165	230	Female
6	미미민지	158	230	Female
7	김노겸	159	240	Female
8	애기공주	198	320	Female
9	집가고싶다고	167	230	Female
10	ㅇ	186	280	male
11	ㅇ[]ㅇ	161	250	Female
12	송재민	184	290	male
13	정세헌	174	270	male
14	권하울	170	270	male
15	박지원	181	300	male
16	류민석	151	220	Female
17	악	156	245	Female
18	이세한	176	265	male
19	ㅇㅅㅇ	162	245	male
20	이연지	165	245	Female
21	어니ㅏㅇ머	162	250	Female
22	윤하준	175	265	male
23	이름	176	270	male

```
In [ ]: df = df.iloc[:,1:]
```

불필요한 데이터 제거

In []: df

	height	shoe size	sex
0	167	250	Female
1	162	245	Female
2	164	255	Female
3	168	240	Female
4	162	240	Female
5	165	230	Female
6	158	230	Female
7	159	240	Female
8	198	320	Female
9	167	230	Female
10	186	280	male
11	161	250	Female
12	184	290	male
13	174	270	male
14	170	270	male
15	181	300	male
16	151	220	Female
17	156	245	Female
18	176	265	male
19	162	245	male
20	165	245	Female
21	162	250	Female
22	175	265	male
23	176	270	male

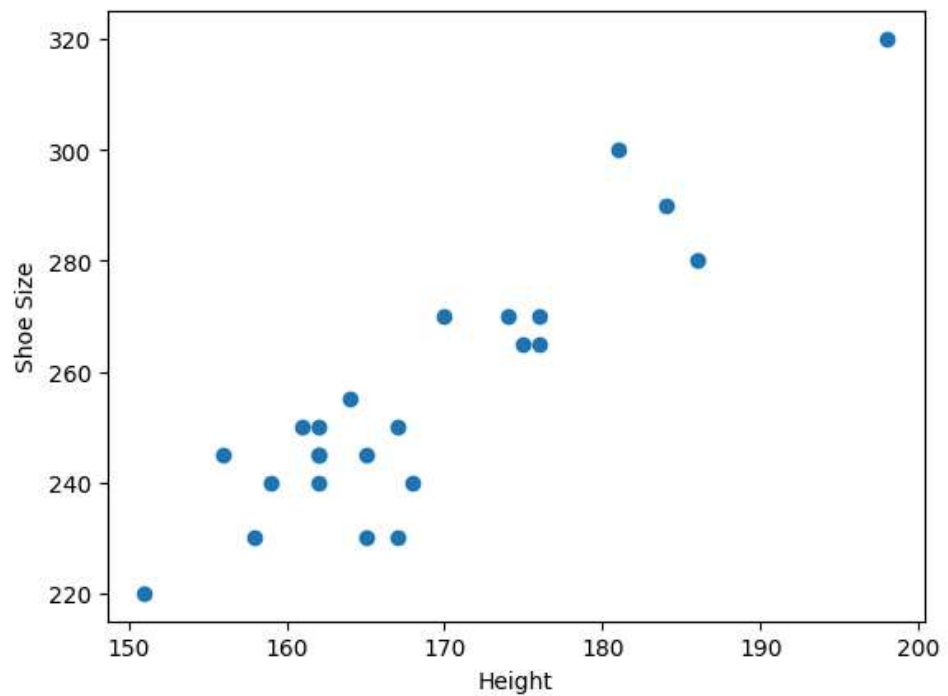
In []:

데이터 시각화

시각화1: 산점도 플롯

```
In [ ]: xs = df['height'].values
ys = df['shoe size'].values
fig,ax = plt.subplots()
ax.plot(xs,ys,'o')
plt.ylabel('Shoe Size')
plt.xlabel('Height')
```

Text(0.5, 0, 'Height')

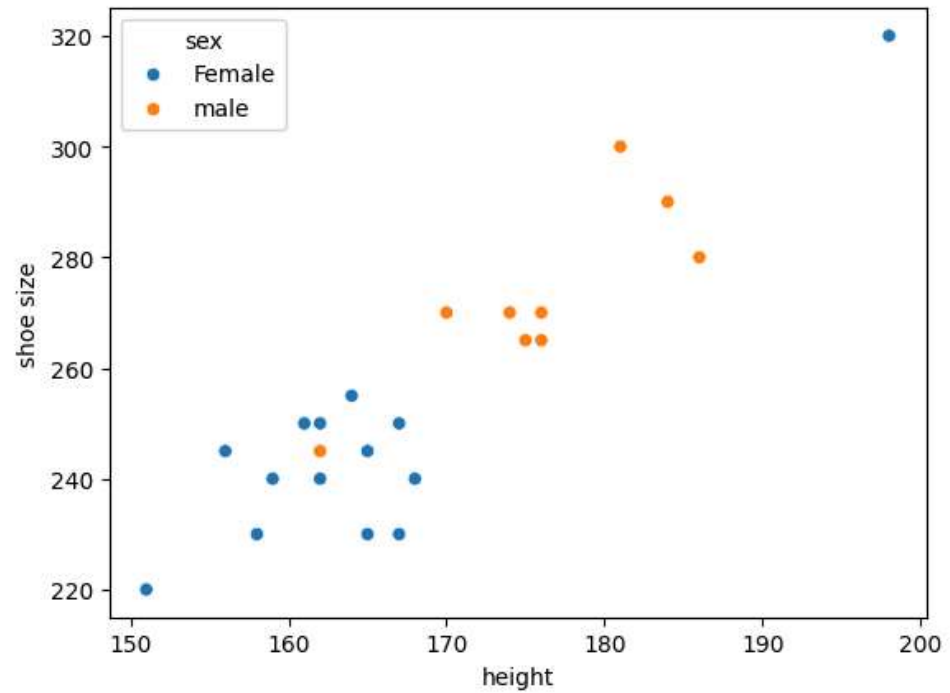


시각화2 = 시각화1 + 성별 정보

-> 좋은 시각화는 데이터를 더 명료하게 보여준다.

```
In [ ]: import seaborn as sns
sns.scatterplot(data=df, x='height', y='shoe size', hue='sex')
```

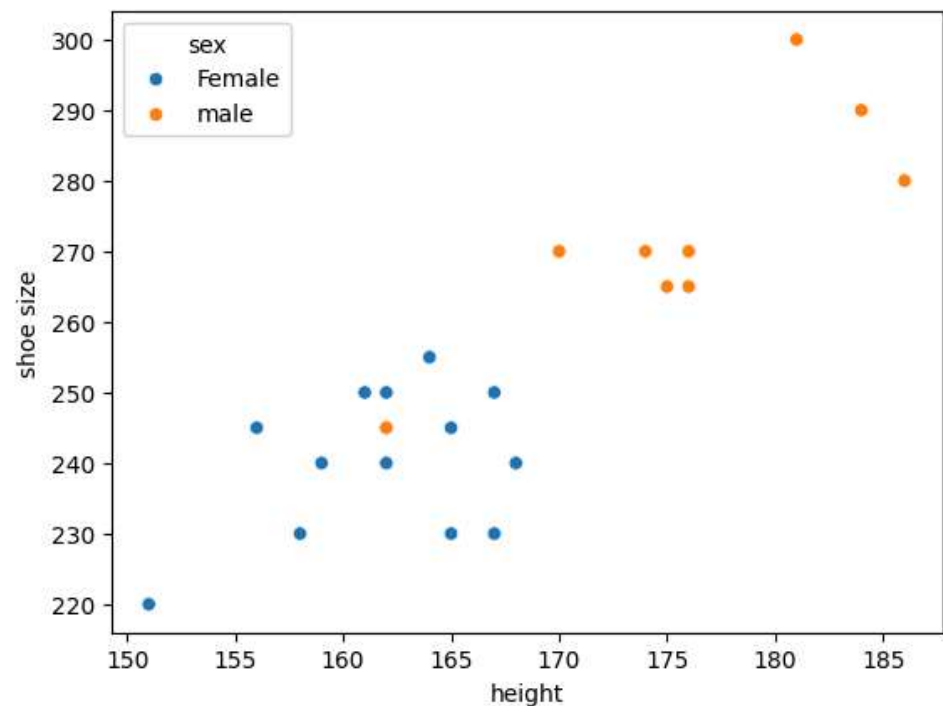
<Axes: xlabel='height', ylabel='shoe size'>



이상치 제거

```
In [ ]: is_outlier = df['height'].values>190
df = df[~is_outlier]
sns.scatterplot(data=df, x='height', y='shoe size', hue='sex')
```

<Axes: xlabel='height', ylabel='shoe size'>




```
In [ ]: import io
import pandas as pd
df_new = pd.read_csv(io.StringIO('''
ㅈㅈ,180,275,Male
대동로이스,178,275,Male
건퍼우,176,290,Male
---,163,230,Female
유우시,175,265,Male
정지웅,176,270,Male
찐주노,175,275,Male
👤,173,275,Male
ㅇ,161,240,Female
h,168,245,Female
,181,290,Male
no,170,265,Male
대동반다이크,175,280,Male
대동벨링엄,170,265,Male
주훈,161,245,Female
배고파,157,220,Female
이재용,182,255,Male
방준호,130,290,Female
방주노,180,280,Male
뽕주노,178,280,Male
?,178,280,Male
;;,160,235,Female
,178,275,Male
주,175,280,Male
'''), header=None)
```

```
In [ ]: df_4 = df
df = df_new
```

```
In [ ]: df_4
```

	height	shoe size	sex
0	167	250	Female
1	162	245	Female
2	164	255	Female
3	168	240	Female
4	162	240	Female
5	165	230	Female
6	158	230	Female
7	159	240	Female
9	167	230	Female
10	186	280	male
11	161	250	Female
12	184	290	male
13	174	270	male
14	170	270	male
15	181	300	male
16	151	220	Female
17	156	245	Female
18	176	265	male
19	162	245	male
20	165	245	Female
21	162	250	Female
22	175	265	male
23	176	270	male

```
In [ ]: df.columns = ['ID','height','shoe size','sex']
```

In []: df_new

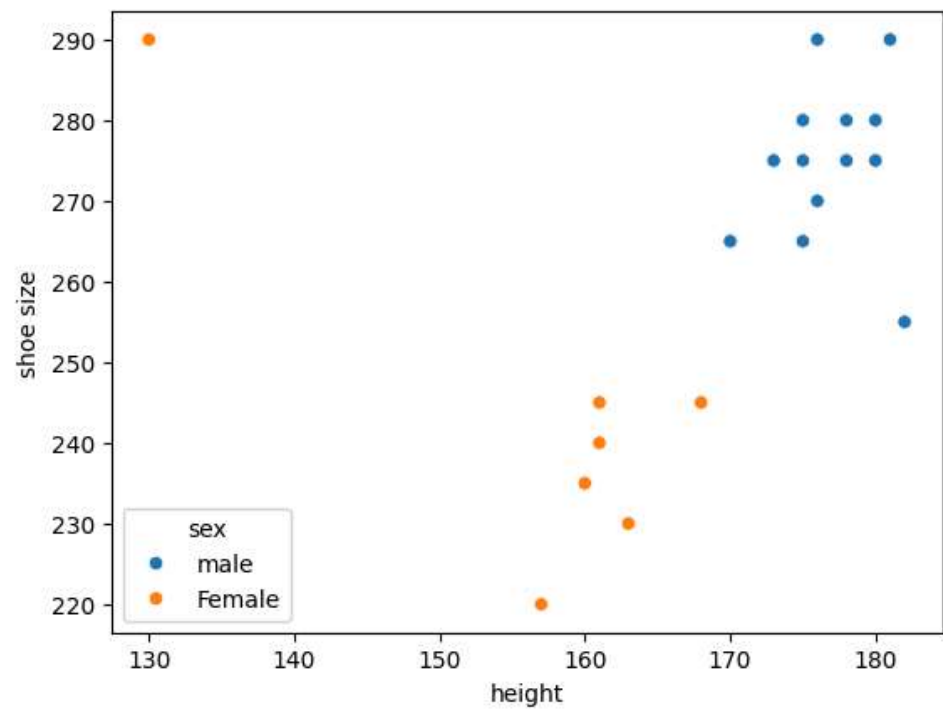
	0	1	2	3
0	ㅈㅈ	180	275	Male
1	대동로이스	178	275	Male
2	건퍼우	176	290	Male
3	---	163	230	Female
4	유우시	175	265	Male
5	정지웅	176	270	Male
6	찐주노	175	275	Male
7	🐼	173	275	Male
8	ㅇ	161	240	Female
9	h	168	245	Female
10	NaN	181	290	Male
11	no	170	265	Male
12	대동반다이크	175	280	Male
13	대동벨링엄	170	265	Male
14	주훈	161	245	Female
15	배고파	157	220	Female
16	이재용	182	255	Male
17	방준호	130	290	Female
18	방주노	180	280	Male
19	빵주노	178	280	Male
20	?	178	280	Male
21	::	160	235	Female
22	NaN	178	275	Male
23	주	175	280	Male

```
In [ ]: gender_map = {
        'Male': 'male',
        'Female': 'Female'
    }
df['sex'] = df['sex'].map(gender_map)
df
```

	ID	height	shoe size	sex
0	ㅈㅈ	180	275	male
1	대동로이스	178	275	male
2	건퍼우	176	290	male
3	---	163	230	Female
4	유우시	175	265	male
5	정지웅	176	270	male
6	찐주노	175	275	male
7		173	275	male
8	o	161	240	Female
9	h	168	245	Female
10	NaN	181	290	male
11	no	170	265	male
12	대동반다이크	175	280	male
13	대동벨링엄	170	265	male
14	주훈	161	245	Female
15	배고파	157	220	Female
16	이재웅	182	255	male
17	방준호	130	290	Female
18	방주노	180	280	male
19	빵주노	178	280	male
20	?	178	280	male
21	;;	160	235	Female
22	NaN	178	275	male
23	주	175	280	male

```
In [ ]: sns.scatterplot(data=df, x='height', y='shoe size',hue='sex')
```

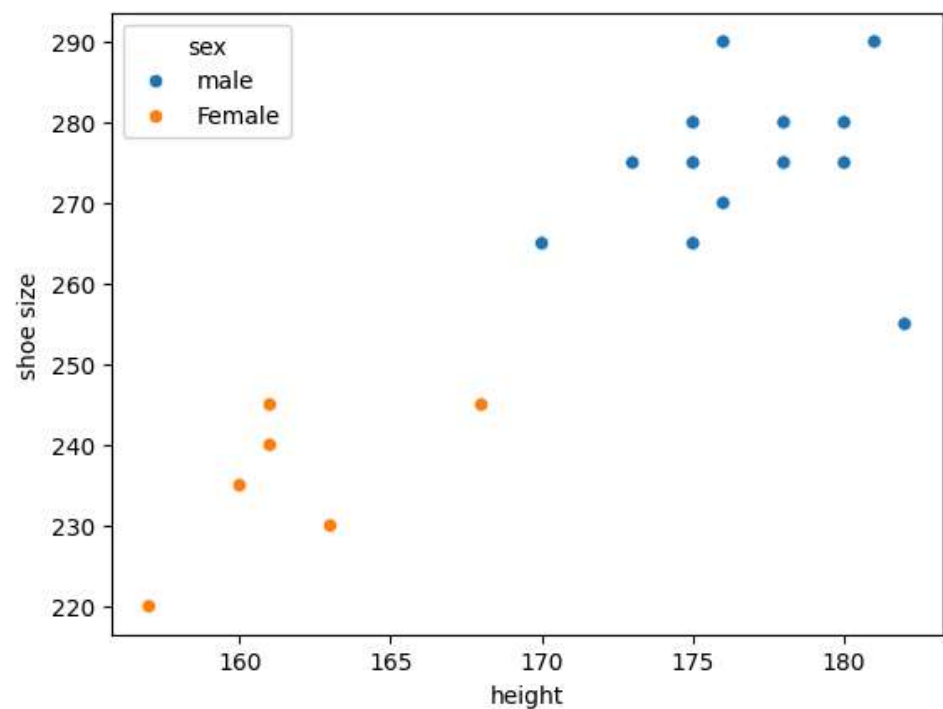
<Axes: xlabel='height', ylabel='shoe size'>



```
In [ ]: sns.scatterplot(data=df, x='height', y='shoe size',hue='sex')
```

```
In [ ]: is_outlier = df['height'].values<140  
df = df[~is_outlier]  
sns.scatterplot(data=df, x='height', y='shoe size',hue='sex')
```

<Axes: xlabel='height', ylabel='shoe size'>

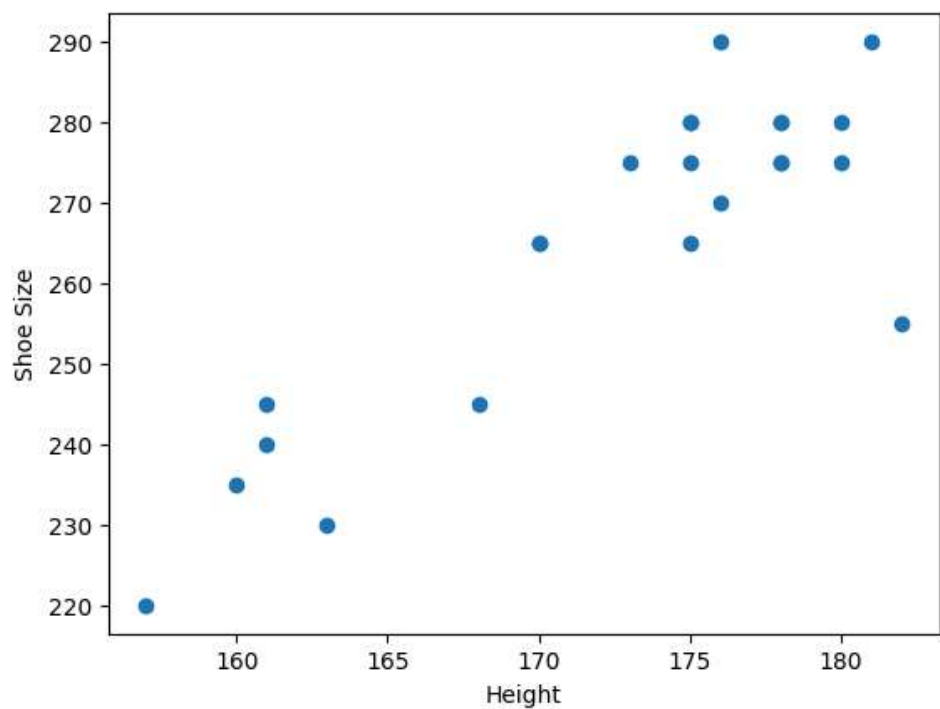


2 선형 회귀

키로 발사이즈 예측하기

```
In [ ]: xs = df['height'].values
ys = df['shoe size'].values
fig, ax = plt.subplots()
ax.plot(xs, ys, 'o')
plt.ylabel('Shoe Size')
plt.xlabel('Height')
```

Text(0.5, 0, 'Height')



##파라미터 튜닝: m,c를 조절해서 적당한 직선 찾기

```

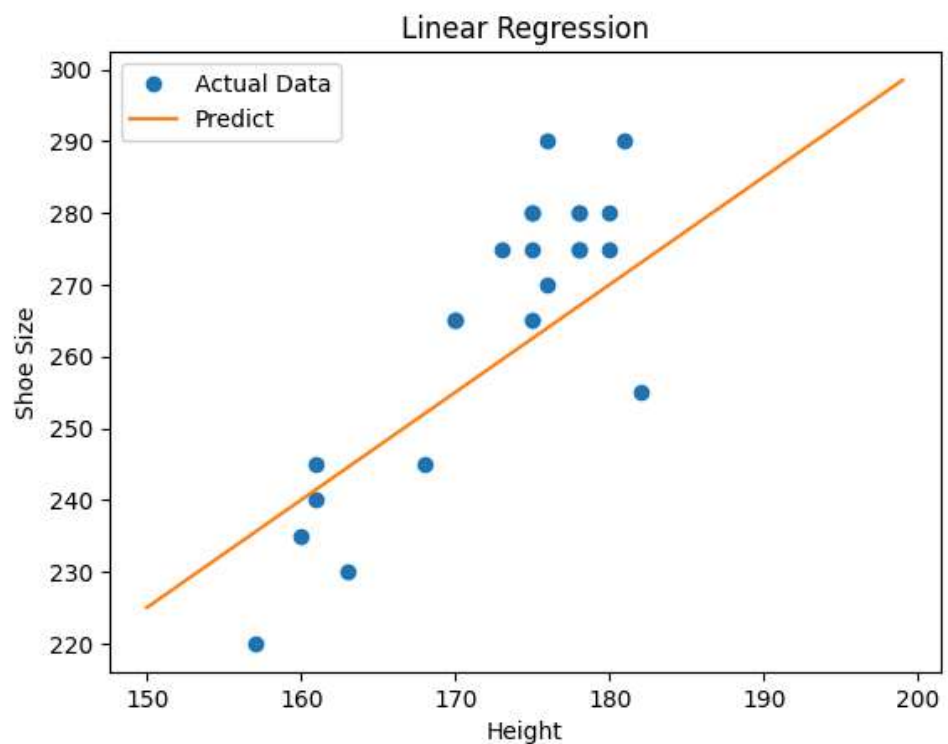
In [ ]: m = 1.5 # @param {type:"raw"}
        c = 0 # @param {type:"raw"}

        fig,ax = plt.subplots()
        ax.plot(xs,ys,'o',label='Actual Data')
        plt.ylabel('Shoe Size')
        plt.xlabel('Height')
        my_figure = fig

        xx = np.arange(150,200)
        plt.plot(xx,m*xx+c,label='Predict')
        plt.legend()
        plt.title('Linear Regression')

```

Text(0.5, 1.0, 'Linear Regression')



다른 친구들은 어떻게 했을까?

##비용함수: 좋은 직선을 수학적으로 정의하기

```

In [ ]: m = 1# @param {type:"number"}
c = 90# @param {type:"number"}
fcn_estimate = lambda x: m*x + c
xs = df['height'].values
ys = df['shoe size'].values
def sort_by_x(x_unsorted, y_unsorted):
    """
    Sorts two lists/arrays based on the values in the first list/array.

    Args:
        x_unsorted: The list or array to sort by.
        y_unsorted: The list or array to sort according to the order of
x_unsorted.

    Returns:
        A tuple containing the sorted x_unsorted and y_unsorted.
    """
    # Combine x and y into pairs, sort by x, then separate
    sorted_pairs = sorted(zip(x_unsorted, y_unsorted))
    x_sorted, y_sorted = zip(*sorted_pairs)
    return np.array(x_sorted), np.array(y_sorted)

# Example usage (optional - can be removed or commented out)
# x_unsorted = [5, 2, 8, 1, 9]
# y_unsorted = [10, 4, 16, 2, 18]
# x_sorted, y_sorted = sort_by_x(x_unsorted, y_unsorted)
# print("Sorted x:", x_sorted)
# print("Sorted y:", y_sorted)
def visualize_MSE(fcn_estimate,xs,ys,l_compare=100):
    l = l_compare
    xs,ys = sort_by_x(xs,ys)
    fcn = fcn_estimate
    fig, axs = plt.subplots(1, 2, layout='constrained',figsize=(10, 5))
    plt.sca(axs[0])
    plt.plot(xs,ys,'o',label='Actual Data')
    x_smooth = np.linspace(xs.min(), xs.max(), 100)
    plt.plot(x_smooth,fcn(x_smooth),label='Predict')
    plt.ylabel('Shoe Size')
    plt.xlabel('Height')
    xs = np.array(xs)
    ys = np.array(ys)
    errors = ys - fcn(xs)
    clr_error = []
    for i, error in enumerate(errors):
        if error >= 0:
            clr = 'red'
        else:
            clr = 'blue'
        plt.plot([xs[i], xs[i]], [fcn(xs[i]), ys[i]], color=clr,
linestyle='-')
        clr_error.append(clr)
    plt.legend()
    plt.title('Visualize error')

```



```

plt.sca(axes[1])
# 정사각형 넓이 리스트
square_areas = errors*errors

# 정사각형 색상 리스트
colors = ['red', 'blue', 'green', 'yellow', 'purple']

# 넓이 합 계산
total_area = sum(square_areas)

# 시각화 시작
ax = axes[1]

# 정사각형 그리기
start_x = 0

for i, area in enumerate(square_areas):
    #area = abs(errors[i])
    side = np.sqrt(area) # 정사각형 한 변의 길이 계산
    rect = plt.Rectangle((start_x, 0), side, side,
facecolor=clr_error[i])
    ax.add_patch(rect)
    start_x += side # 다음 정사각형 시작 위치 설정

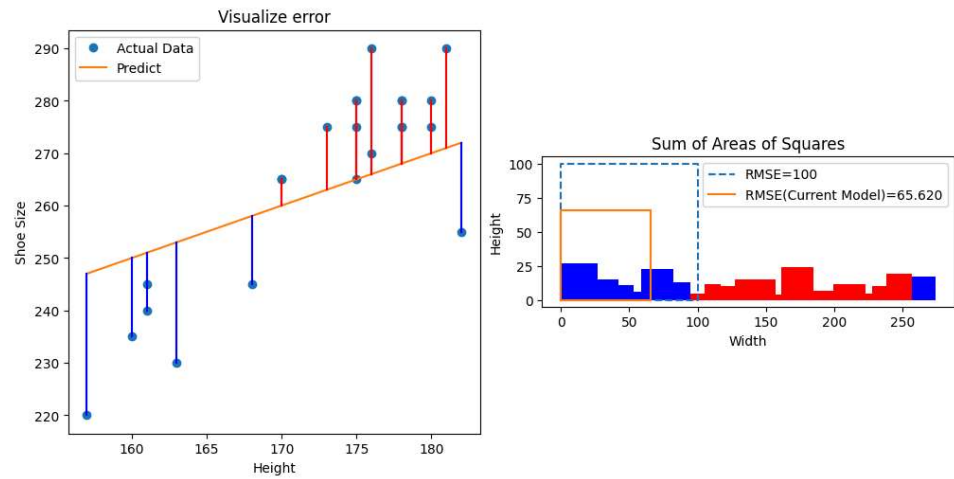
# 텍스트 추가 (넓이 합 표시)
#plt.text(0.5, -0.1, f'Total Area: {total_area}',
#        horizontalalignment='center', verticalalignment='center',
#        transform=ax.transAxes, fontsize=12)

# 그래프 설정

#ax.set_xlim([0, 100])
#ax.set_ylim([0, 100]) # y축 범위 설정
ax.set_aspect('equal') # 정사각형 모양 유지
plt.title('Sum of Areas of Squares')
plt.xlabel('Width')
plt.ylabel('Height')
plt.plot([0, 1, 1, 0, 0],[0, 0, 1, 1, 0], '--',label=f'RMSE={l}')
lsum = np.sqrt(square_areas.sum())
xs_sum = np.array([0,1,1,0,0])
ys_sum = np.array([0,0,1,1,0])
plt.plot(xs_sum*lsum,ys_sum*lsum,label=f"RMSE(Current Model)=
{lsum:.3f}")
plt.legend()

visualize_MSE(fcn_estimate,xs,ys)

```



x와 y값을 넣어주면 자동으로 m과 c를 정해줄 수는 없을까?

3 로지스틱 회귀

시그모이드랑 친해지기

$$\sigma(x) = \frac{1}{1 + e^{-(Wx + b)}}$$

<https://www.desmos.com/calculator/zxl48byttf>

발사이즈로 성별 예측하기

```
In [ ]: df['is_male'] = (df['sex'].values == 'male')
```

```
In [ ]: sns.scatterplot(data=df, x='shoe size', y='is_male', alpha=.3)
```

##파라미터 튜닝

```
In [ ]: W=1# @param {type:"number"}
b=-250# @param {type:"number"}
xs = df['shoe size'].values
ys = df['is_male'].values
def sigmoid(x, W, b):
    return 1 / (1 + np.exp(-(W * x + b)))
visualize_MSE(lambda x:sigmoid(x, W, b),xs,ys,l_compare=1)
```

숙제

예측 문제를 함수 찾기라는 수학적 상황으로 바꾸어 생각할 수 있었다.
그리고 비용함수라는 것을 도입해 좀 더 효율적으로 함수를 찾을 수 있었다.

- Q1. 최적값을 찾는 과정을 컴퓨터에게 시킬 수는 없을까?
- Q2. 우리는 비용함수로 MSE(오차제곱평균)을 썼다. 더 좋은 비용함수는 없을까?
- Q3. 성별을 예측할 때, 키로 예측하는 것과 발로 예측하는 것 둘 중 뭐가 더 좋을까?

다음 수업 예고

```
In [ ]: import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np

# 1. MNIST 데이터셋 불러오기
(train_images, train_labels), (test_images, test_labels) =
tf.keras.datasets.mnist.load_data()

print(f"훈련 이미지 개수: {len(train_images)}")
print(f"테스트 이미지 개수: {len(test_images)}")
print(f"훈련 이미지 형태: {train_images.shape}")
print(f"훈련 라벨 형태: {train_labels.shape}")

# 2. 전체적인 이미지 표본 보여주기
plt.figure(figsize=(10, 10))
for i in range(25):
    plt.subplot(5, 5, i + 1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(train_images[i], cmap=plt.cm.binary)
    plt.xlabel(train_labels[i])
plt.suptitle("MNIST Dataset Samples", fontsize=16)
plt.show()

# 3. 인덱스에 따른 이미지 조회 함수
def display_mnist_image(index, dataset_type='train'):
    if dataset_type == 'train':
        images = train_images
        labels = train_labels
        data_name = "Train"
    elif dataset_type == 'test':
        images = test_images
        labels = test_labels
        data_name = "Test"
    else:
        print("Invalid dataset_type. Choose 'train' or 'test'.")
        return

    if 0 <= index < len(images):
        plt.figure(figsize=(4, 4))
        plt.imshow(images[index], cmap=plt.cm.binary)
        plt.title(f"{data_name} Image Index: {index}, Label:
{labels[index]}")
        plt.xticks([])
        plt.yticks([])
        plt.show()
    else:
```

```
print(f"Index out of bounds. For {data_name} dataset, index must  
be between 0 and {len(images) - 1}.")
```

Downloading data from

[https://storage.googleapis.com/tensorflow/tf-keras-](https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz)
[datasets/mnist.npz](https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz)

←[1m11490434/11490434←[0m ←[32m—————←[0m←[37m←[0m

←[1m0s←[0m 0us/step

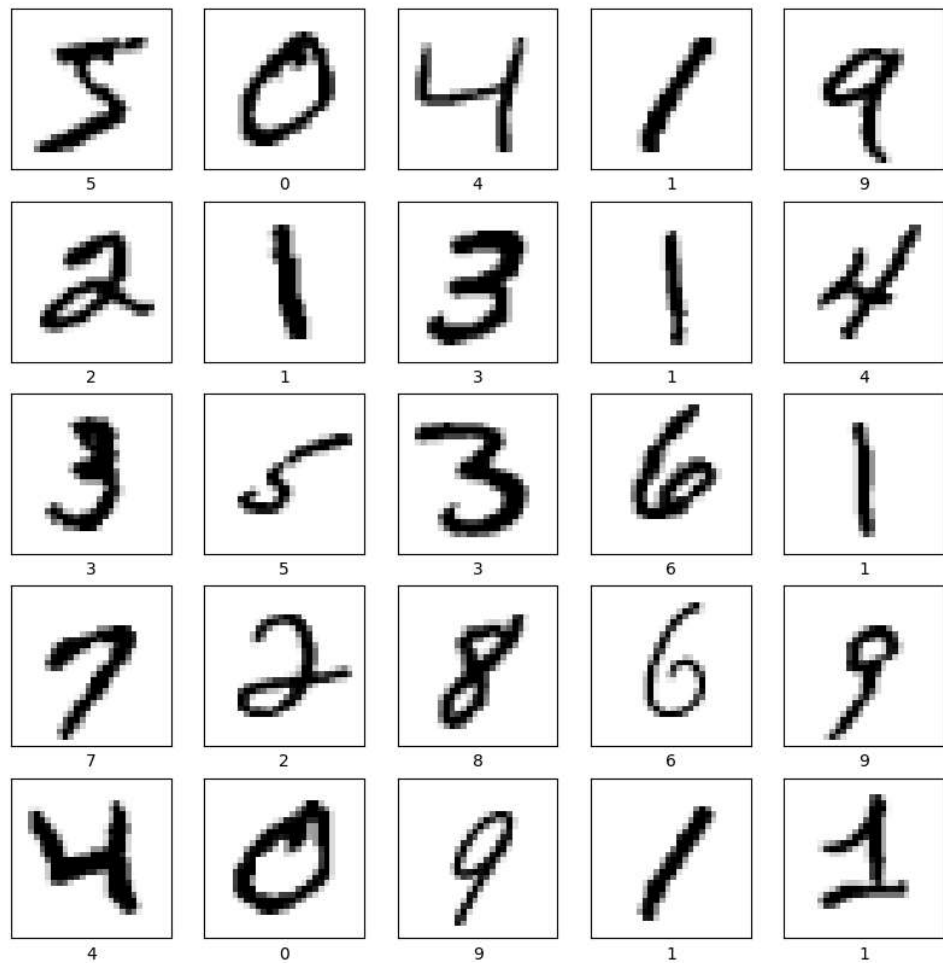
훈련 이미지 개수: 60000

테스트 이미지 개수: 10000

훈련 이미지 형태: (60000, 28, 28)

훈련 라벨 형태: (60000,)

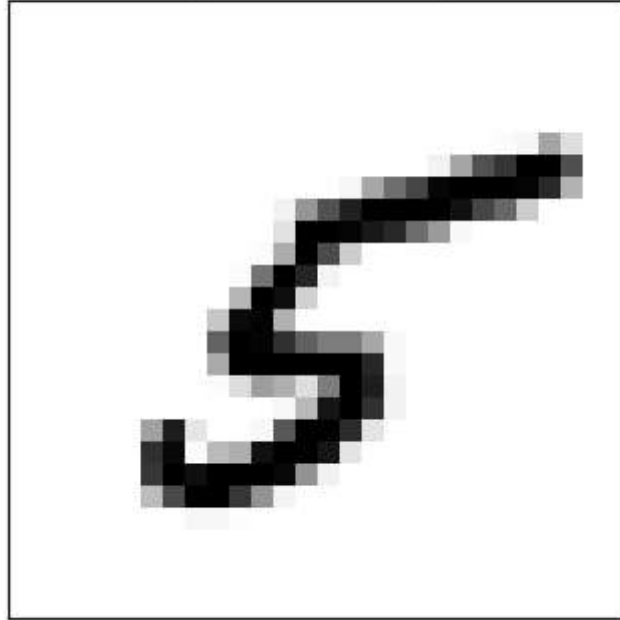
MNIST Dataset Samples



```
In [ ]: print("\n--- 이미지 인덱스 조회 예시 ---")
# 예시: 훈련 데이터셋의 10번째 이미지 조회
display_mnist_image(2401, dataset_type='train')
```

--- 이미지 인덱스 조회 예시 ---

Train Image Index: 2401, Label: 5



사람이 손글씨 0-9까지 라벨되어있는 데이터가 있다. 우리는 손글씨 예측을 자동화하고 싶다.

- 1) 데이터를 어떻게 수학적으로 표현할지
- 2) 어떤 예측함수를 쓸지
- 3) 어떤 손실함수를 쓸지
